

PSP · Ejercicios de ProcessBuilder y Process

Normas

- Captura o declara las excepciones `IOException` e `InterruptedException`.
- Muestra mensajes claros por pantalla que expliquen qué está pasando en cada paso.
- Responde por escrito a las preguntas marcadas como *Reflexión*.

Bloque 1 · Lanzar procesos

Ejercicio 1. Mi primer proceso

Escribe un programa `Ejercicio1` que abra la Calculadora con el comando `open -a Calculator`. Después, modifícalo para que abra `TextEdit`.

Ejercicio 2. ¿Quién es quién?

Escribe un programa `Ejercicio2` que lance la Calculadora y muestre por pantalla:

- el PID del proceso lanzado;
- el PID del propio programa Java.

Reflexión: añade al final `Thread.sleep(30000)` para que el programa no termine enseguida. Mientras espera, busca los dos PID en la terminal con `ps -ef | grep java` y `ps -ef | grep Calculator`. ¿Coinciden con los que muestra tu programa? ¿Es el proceso de la Calculadora el que has lanzado tú?

Ejercicio 3. Lanzador por argumentos

Escribe un programa `Ejercicio3` que reciba por `args` un comando y sus argumentos, y lo lance. Por ejemplo:

```
java -cp bin paquete.Ejercicio3 open -a Calculator
```

- Antes de lanzarlo, muestra el comando completo.
- Si no se pasa ningún argumento, muestra un mensaje explicando cómo se usa el programa y termina.

Bloque 2 · Esperar y códigos de salida

Ejercicio 4. ¿Ha ido bien?

Escribe un programa `Ejercicio4` que lance el comando `mkdir pruebas` y espere a que termine. Después:

- muestra el código de salida del proceso;
- si es 0, muestra "Carpeta creada"; en otro caso, "Error al crear la carpeta".

Ejecuta el programa dos veces seguidas.

Reflexión: ¿qué código de salida obtienes la segunda vez? ¿Por qué?

Ejercicio 5. Comprobador de conexión

Escribe un programa `Ejercicio5` que pida al usuario un nombre de host con `Scanner`, lance `ping -c 1 <host>` y, según el código de salida, indique si el host responde o no.

Pruébalo con `google.com` y con `noexiste.invalid`.

Ejercicio 6. Trabajar en otra carpeta

Escribe un programa `Ejercicio6` que cree un fichero vacío con el comando `touch fichero.txt`, pero **dentro de la carpeta pruebas** del ejercicio 4, sin escribir la ruta en el comando.

Cuando el proceso termine, comprueba desde Java con `File.exists()` que el fichero se ha creado en el sitio correcto.

Bloque 3 · Varios procesos a la vez

Ejercicio 7. Secuencial frente a paralelo

Escribe un programa `Ejercicio7` que lance tres procesos: `sleep 3`, `sleep 5` y `sleep 1`. Haz dos versiones:

- **Versión A (secuencial):** lanza un proceso, espera a que termine y lanza el siguiente.
- **Versión B (paralela):** lanza los tres procesos y, después, espera a que terminen los tres.

Mide el tiempo total de cada versión con `System.currentTimeMillis()` y muéstralo en segundos.

Reflexión: ¿cuánto tarda cada versión? ¿De qué depende el tiempo total en cada caso?

Ejercicio 8. Java lanza Java

Crea dos clases:

- `Hijo`: recibe un número por `args` y termina con `System.exit(numero)`.
- `Padre`: lanza tres procesos `Hijo` con los valores 0, 5 y 42, espera a cada uno y muestra el código de salida que ha devuelto.

Pista: el comando para lanzar el hijo es como este (cambia `paquete` por el nombre de tu paquete):

```
String[] comando = {"java", "-cp", "bin", "paquete.Hijo", "5"};
```

Ejecuta `Padre` desde la raíz del proyecto, para que la ruta `bin` sea correcta.

Reflexión: el código de salida es una forma de que el hijo comunique un resultado al padre. ¿Qué limitaciones tiene? Prueba con `System.exit(300)` y con `System.exit(-1)`: ¿qué código recibe el padre?

Bloque 4 · Controlar y terminar procesos

Ejercicio 9. Tiempo límite

Escribe un programa `Ejercicio9` que lance `sleep 10` y espere como máximo 3 segundos a que termine. Si pasado ese tiempo sigue vivo, destrúyelo.

Muestra por pantalla:

- si el proceso está vivo antes de destruirlo;
- si está vivo después de destruirlo;
- su código de salida final.

Reflexión: ¿qué código de salida obtienes? Busca qué significa la señal `SIGTERM` y relaciónala con ese número.

Ejercicio 10. El proceso rebelde

Escribe un programa `Ejercicio10` que lance un proceso que **ignora** la petición de terminar:

```
ProcessBuilder pb = new ProcessBuilder("sh", "-c", "trap '' TERM; sleep 30");
```

1. Intenta terminarlo con `destroy()` y espera como máximo 2 segundos. Muestra si ha terminado.
2. Si no ha terminado, térmalo con `destroyForcibly()` y muestra su código de salida.

Reflexión: ¿qué diferencia hay entre pedir a un proceso que termine y obligarle a terminar? ¿Por qué el código de salida es distinto al del ejercicio 9?

Ejercicio 11. Árbol de procesos

Escribe un programa `Ejercicio11` que lance esta estructura de procesos (un padre, dos hijos y un nieto):

```
ProcessBuilder pb = new ProcessBuilder("sh", "-c", "sh -c 'sleep 60 & wait' & sleep 60 & wait");
```

Espera medio segundo para que se creen todos los procesos y, después:

1. Muestra cuántos procesos devuelven `children()` y `descendants()`.
2. Para cada descendiente, muestra su PID, el PID de su padre y su comando.
3. Destruye todos los descendientes y el proceso padre, y comprueba que no queda ninguno vivo.

No uses expresiones lambda: recorre los procesos con un `Iterator`.

Reflexión: ¿qué diferencia hay entre `children()` y `descendants()`? Dibuja el árbol de procesos con sus PID.

Bloque 5 · Reto final

Ejercicio 12. Mini gestor de procesos

Escribe un programa `GestorProcesos` que muestre este menú en bucle hasta que el usuario elija salir:

1. Lanzar proceso
2. Listar procesos lanzados
3. Terminar un proceso por PID
4. Salir

- **Opción 1:** pide un comando completo (por ejemplo, `sleep 100`), lo lanza y guarda el `Process` en un `ArrayList`. Muestra el PID del proceso lanzado.
- **Opción 2:** para cada proceso lanzado, muestra su PID, su comando y si sigue vivo. Si ya ha terminado, muestra también su código de salida.
- **Opción 3:** pide un PID y termina ese proceso. Si no termina en 2 segundos con `destroy()`, usa `destroyForcibly()`. Si el PID no corresponde a ningún proceso lanzado, muestra un aviso.
- **Opción 4:** antes de salir, termina todos los procesos que sigan vivos.

Ampliación: añade una opción que termine **cualquier** proceso del sistema a partir de su PID, aunque no lo haya lanzado el gestor, usando `ProcessHandle.of(pid)`. Pruébalo con un `sleep 300` lanzado desde la terminal.

Chuleta de métodos

Clase	Método	Qué hace
<code>ProcessBuilder</code>	<code>new ProcessBuilder(String...)</code>	Prepara un comando y sus argumentos
<code>ProcessBuilder</code>	<code>command(String...) / command()</code>	Cambia o consulta el comando
<code>ProcessBuilder</code>	<code>directory(File)</code>	Fija la carpeta de trabajo del proceso
<code>ProcessBuilder</code>	<code>start()</code>	Lanza el proceso y devuelve un <code>Process</code>
<code>Process</code>	<code>pid()</code>	PID del proceso

<code>Process</code>	<code>waitFor()</code>	Espera a que termine y devuelve su código de salida
<code>Process</code>	<code>waitFor(long, TimeUnit)</code>	Espera como máximo ese tiempo; devuelve <code>true</code> si ha terminado
<code>Process</code>	<code>exitValue()</code>	Código de salida; lanza una excepción si aún no ha terminado
<code>Process</code>	<code>isAlive()</code>	<code>true</code> si el proceso sigue en ejecución
<code>Process</code>	<code>destroy()</code>	Pide al proceso que termine (SIGTERM)
<code>Process</code>	<code>destroyForcibly()</code>	Obliga al proceso a terminar (SIGKILL)
<code>Process</code>	<code>children() / descendants()</code>	Hijos directos / todos los descendientes
<code>ProcessHandle</code>	<code>ProcessHandle.current()</code>	El proceso actual (tu programa Java)
<code>ProcessHandle</code>	<code>ProcessHandle.of(pid)</code>	Cualquier proceso del sistema a partir de su PID
<code>ProcessHandle</code>	<code>parent()</code>	Proceso padre
<code>ProcessHandle</code>	<code>info().commandLine()</code>	Comando con el que se lanzó el proceso